Replicating the SNAP Error-Ranking Work for Your State

October 23, 2026 · Replication guide · Cory Trimm

Who this is for

This guide is for a state SNAP team that wants to know what the error-ranking tool would do for their state, and then try it. It assumes one analyst comfortable running Python scripts, and a program or QC lead who can answer questions about case data and review practice. No machine-learning background is needed for levels 1 and 2.

The work behind it is described in the *Project Summary* and in short briefs for Pennsylvania, New York and Wyoming. The code is open: github.com/ctrimm/snap-qc-per-experiments.

What you get, in four levels:

Level	Time	What you get
1. Look up your state	An hour	Your state's measured results and cost-share scenarios from the published models
2. Reproduce from public data	An afternoon	The same numbers, recomputed on your machine from the federal files
3. Score your own cases	A few days	A ranked review queue for your current caseload, using the packaged model
4. Pilot and refit	Three to six months	Measured dollars found in your state, and a model refit on your own data

In plain terms: the tool reads a few fields your agency already records for every active case (income, deductions, housing costs, household size, the benefit paid). From those it estimates how many error dollars each case is likely to hold. Reviewers start at the top of that list instead of choosing at random. Across all 53 jurisdictions, the top fifth of the list held a median of four times the error dollars of a random fifth. Nothing about a household is decided by the tool; it only orders the review queue.

What you need

- **A computer:** any laptop with Python 3.10 or newer. No GPU and no cloud service.
- **Software:** the repository plus numpy, joblib and scikit-learn 1.9.1 (exactly that version: the saved models are pickles tied to it, and other versions may refuse to load them or score differently).
- **Public data (levels 2–4):** the FY2020–2024 SNAP QC public-use CSV files from snapqcdata.net/datafiles, unzipped beside the repository.
- **Your state's data (levels 3–4):** an extract of active cases with the fields in *Level 3, Step 1*, and later your own review findings.

```
git clone https://github.com/ctrimm/snap-qc-per-experiments.git
cd snap-qc-per-experiments
python3 -m pip install numpy joblib scikit-learn==1.9.1
# data layout the code expects (a sibling of the repository):
#   ../snap-qc-data/raw/qcfy2020_csv/qc_pub_fy2020.csv  ...  ../snap-qc-
data/raw/qcfy2024_csv/qc_pub_fy2024.csv
```

Level 1: look up your state

Every jurisdiction's results are committed in the repository. Replace `XX` with your state's USPS code:

```
# held-out results of your state's deployed model: lift, $ per review hour, forecast
check, fairness
python3 -c "import json;
print(json.dumps(json.load(open('experiments/exp83_results.json'))['XX'], indent=1))"
# FY2028 cost-share scenarios, every step shown
python3 experiments/cost_share_replicate.py --state XX
```

How to read the first output:

Field	Meaning
`top20.lift`	Error dollars found in the top 20% of the queue, as a multiple of random review. Typical: 3–4.5
`top10.lift`	The same for the top 10%. Typical: 5–7

Field	Meaning
`top20.per_hour_$` (`random_per_hour_$`)	Error dollars found per review hour, assuming 30-minute reviews
`top20.expected_over_actual`	The model's dollar forecast divided by actual dollars; 1.0 is perfect
`fairness.baseline_ratio`	Each household type's share of reviews ÷ its share of error dollars, ranking by dollars only (1.0 = proportionate)
`fairness.floor_ratio`, `floor_lift`	The same with the fairness floor on, and the lift that remains
`variant`	`B_ctx`: pooled with every state plus state context; `C_ctx`: the same with your own cases weighted ten times

The cost-share script reads your state's official FY2025 error rate and annual benefits, then shows the low, central and high scenarios. It gives the rate after ranked reviews, the share owed, dollars avoided, and the reviews and FTEs needed. To try your own assumptions:

```
python3 experiments/cost_share_replicate.py --state XX --per 8.7          # a newer
error rate
python3 experiments/cost_share_replicate.py --state XX --R 0.015          # review 1.5%
of cases a month
python3 experiments/cost_share_replicate.py --state XX --benefits 2.1e9   # current
annual benefits
```

Find your break-even review rate. Savings happen only when the rate drops below a bracket line (6%, 8%, 10%). Raise `--R` until the central line's share drops: that's how much review it takes to cross the next line. `COST-SHARE-REPLICATE.md` explains the model and every parameter.

Level 2: reproduce from public data

This refits every jurisdiction's deployed model from the federal files and checks it against the committed results:

```
python3 experiments/exp83_context_models_full_eval.py --workers 8
git diff --stat experiments/exp83_results.json     # empty = exact reproduction
```

What the script does:

- **Data split:** for each jurisdiction it trains on 75% of that state's QC cases pooled with all other states' cases, and scores the 25% set aside. The held-out cases are fixed in `experiments/exp68_holdouts.json`, so everyone gets the same split.
- **Models:** two gradient-boosted models, one for the chance of an error and one for its dollars. Expected dollars are calibrated by isotonic regression on out-of-fold predictions.
- **Reproducibility:** results are deterministic.

On a Mac Studio (M3 Ultra) it took about 8 minutes with 8 workers (about 2 CPU-hours in all) and reproduced `exp83_results.json` byte for byte on October 23, 2026. A laptop with 4 cores would take roughly half an hour.

Cost-share inputs too:

```
python3 experiments/exp93_cost_share_current_models.py --workers 6   # refits and
measures lifts with the floor; stops unless all match
python3 experiments/cost_share_replicate.py
```

Checking how a state's model was chosen: the adoption rule is in `experiments/exp82_context_all_states.py` (41 larger jurisdictions) and `exp80_state_context.py` (12 small ones). A pooled variant replaced the state-only model only if it was better on both held-out cases and a later-years test (train 2020–22, test 2023–24). The results are in `exp80_results.json` and `exp82_results.json`.

Level 3: score your own cases

Step 1: build the input file

One row per active case, with these columns (QC public-use names). Blank values count as 0.

Column	What it holds
`CERTHHSZ`	Certified household size (people in the SNAP unit)

Column	What it holds
RAWHSIZE	Number of people in the household
RAWGROSS	Gross countable monthly income
RAWNET	Net monthly income after deductions
RAWERND	Earned income deduction (20% of earnings)
SHELDED	Excess shelter deduction
RENT	Rent or mortgage
UTIL	Utility costs
SUA1	Standard utility allowance code; codes 3–8 mean an SUA was used
FSMEDEXP	Medical expense deduction
FSDEPDED	Dependent care deduction
FSCSEXP	Child support deduction
HOMEDED	1 if the homeless shelter deduction was used
RAWBEN	Monthly benefit issued
BENMAX	Maximum allotment for the household's size

Add any case ID column you want carried through. All fields are monthly dollar amounts or counts as recorded **before** any review. Never include review findings or corrected values. The QC technical documentation in the data download defines each field exactly, and your eligibility system's field definitions may differ slightly. Note the differences.

Step 2: score

```
python3 deliverables/state_rankers/score_cases.py --state XX --input cases.csv --output
ranked.csv --id-col CASE_ID
```

The output adds four columns:

Column	Meaning
`p_error`	Chance the case has an error
`expected_dollars`	Expected error dollars, calibrated
`rank`	Position in the review queue
`in_top_pct`	Which top bucket the case is in: 10, 20, 30, 50 or 100

The script also prints the total expected error dollars in the top 20%. Quote it with a ±15–20% band.

Step 3: sanity checks before anyone reviews

- **Spot-check the inputs:** compare the distribution of `RAWBEN`, `RAWGROSS` and household size with your caseload reports. A field mapped to the wrong unit (annual instead of monthly, cents instead of dollars) is the most common failure.
- **Look at the top 50 cases with a QC lead.** Do they look like cases that deserve review?
- **Turn on the fairness floor.** Check each household type's share of the top of the queue against its share of the caseload (see *Fairness* below).

About the packaged models: each `deliverables/state_rankers/XX/model.joblib` was refit on all five years of that state's cases plus every other state's, for the best possible model. Its held-out performance is the evaluation fit in `exp83_results.json`. The `evaluation` block inside `model_card.json` is from the earlier state-only model; use `exp83_results.json`.

Level 4: pilot and refit

A three-month shadow pilot

1. **Each month,** score active cases and give reviewers a sample from the top of the queue alongside their usual selection.
2. **Keep about 10% of reviews on randomly chosen cases.** This is your control, and it shows the model's blind spots.

3. **For every review, record:**
    - error yes or no;
    - over or under, and the dollars;
    - which part of the case (element);
    - roughly when the underlying change happened.
4. **Compare:**
    - dollars found per review hour, top of queue vs random slice vs usual selection;
    - dollars found vs `expected_dollars`;
    - each household type's share of reviews vs its share of errors.

What to report

These are the two numbers that move the cost-share estimate most. Rerun `cost_share_replicate.py --state XX --lift <your measured lift>` with them in mind:

Measure	How	Why it matters
Lift	Error dollars per review, top of queue ÷ random slice	The tool's actual value in your state
Error lifetime (m)	How long found errors had lasted, and would have lasted until recertification	Drives the cost-share range more than the lift
Fix rate (f)	Share of found errors corrected going forward	Same
Forecast accuracy	Σ expected ÷ Σ actual dollars, top 20%	Whether you can budget from the forecast
Fairness	Review share ÷ error share by household type	Whether the floor is working

Fairness

Ranking purely by dollars under-reviews elderly households in 51 of 53 jurisdictions, and disabled households in many. The fairness floor (exp72) guarantees each household type a

share of reviews in line with its share of errors. It costs a median of about 1% of the lift nationally; 3–6% in Pennsylvania and New York; 13% in Wyoming. Keep it on by default.

Underpayments

The main ranker mostly finds overpayments. A second ranker trained on underpayments (exp79) finds a median of 3.4 times the underpayment dollars of random review, against 1.5 times for the main ranker, and does better in every jurisdiction. States with many underpayments (Wyoming: 54% of error dollars) should run both queues.

Refitting on your own data

Your own review findings, from the pilot and from routine second-party reviews, can improve the model. Two rules, from a simulation over four years in six states (exp67):

- **Never retrain only on the cases the model flagged.** It stops learning about errors it doesn't flag, and lost about 15% of its edge in four years in that simulation. Always include the yearly federal QC sample (random and fully reviewed) and the pilot's random slice.
- **Refresh once a year,** when the new QC public-use file is released.

The code change: the training code reads cases in the QC public-use layout (`experiments/exp58_agency_rankers.py`, `load_state`; the year list `FY_FILES`). The input fields above plus `STATUS` (1 correct, 2 overpaid, 3 underpaid) and `AMTERR` (error dollars) are all it needs. Export your reviewed cases in that layout as an extra year's file, add it to `FY_FILES`, and rerun the fit for your state. This is a small change. Keep the federal years in, and hold out a recent slice of your own cases to test on before replacing the packaged model.

Flag cards (optional)

A flag card tells the reviewer why a case was flagged and which parts to check first. Its top three items included the real error for 86–88% of flagged errors in Pennsylvania and New York. The card logic is in `experiments/exp62_*`, `exp65_*` and `exp74_*`. The "check first" model needs about 100–200 labelled error cases from your state to beat generic advice (exp63), which a pilot produces in a few months.

Your state's policy library (optional)

Rule citations on flag cards, and any policy-lookup assistant, read from **state-policy-docs** (github.com/ctrimm/state-policy-docs). It is thorough for federal rules and California but thin for most states (Pennsylvania 4 documents, New York 6, Wyoming 2).

To add your state's manual: follow `AGENTS.md` in that repository (`pipeline/ingest.py`). Each document becomes a Markdown file with front matter for jurisdiction and effective dates. Then link citations, check the links, and run the tests:

```
.venv/bin/python pipeline/link_citations.py
.venv/bin/python pipeline/check_links.py
.venv/bin/python -m unittest discover -s tests
```

One lesson from California: keep superseded or older guidance reachable by citation rather than in search (`STATE_POLICY_CITED_LETTERS=cite_only`). Mixing hundreds of old letters into search pushed current rules out of the top results.

Checklist

Step	Done when
Level 1	You have your state's `exp83_results.json` entry and the cost-share output, and know your break-even review rate
Level 2	`exp83_context_models_full_eval.py` reproduces `exp83_results.json` with an empty `git diff`
Level 3	`score_cases.py` ranks a month of your cases, the inputs pass the sanity checks, and a QC lead has looked at the top 50
Level 4	Three months of shadow reviews with a random slice; lift, forecast accuracy, error lifetime, fix rate and fairness measured; cost share rerun with your lift
Refresh	Yearly refit including the new federal QC sample and your random-slice reviews

Cautions

- **The tool prioritizes; it doesn't decide.** Every determination about a household stays with a person.
- **Cost-share figures are scenarios, not forecasts.** They leave out:
    - timing relative to the federal sample month;
    - procedural errors outside the public file;
    - behavior changes;
    - the delay option;
    - the cost of reviews.
- **Small states:** with fewer than about 1,500 QC cases (Alaska, Guam, the Virgin Islands, Wyoming), held-out results are less certain. Check them against your own reviews before quoting forecasts.
- **Never touch the federal QC sample.** The tool is for a state's own reviews of active cases; the federal sample stays random by law.
- **`model.joblib` files are pickles.** Load only copies from this repository.